KOI Integration

for Cortex XSIAM

Customer User Guide

Prepared for: **[CUSTOMER NAME]**

Document	KOI Content Pack — Customer User Guide
Customer	[CUSTOMER NAME]
Pack version	1.5.0
Guide revision	1.4.0
Applies to	Cortex XSIAM — event collection, commands, playbooks and dashboard
Last updated	July 24, 2026

Table of Contents

1. Introduction

1.1 About KOI

KOI is an endpoint security platform that provides visibility and control over the software your workforce actually runs: browser extensions, IDE plugins, code and OS packages, desktop applications, and the emerging agentic AI surface — MCP servers, AI models, skills, and AI-powered tools. KOI discovers these items across your endpoints, scores their risk, enforces allow/block policies, and generates alerts and audit events.

1.2 What is in this content pack

Component	Purpose
KOI integration	Event collector (Alerts + Audit logs into Cortex XSIAM) plus 26 commands for inventory, devices, governance, allow/block lists, and the Koidex catalog.
Koi Parsing Rule	Normalizes raw KOI events arriving in the koi_koi_raw dataset and promotes alert and audit fields for modeling.
Koi Modeling Rule	Maps KOI alert and audit events to the Cortex Data Model (XDM).
Koi Alerts Dashboard	Ready-made XSIAM dashboard visualizing KOI alert activity.
KOI Script Runner playbooks	Five playbooks plus the KoiScanTracker automation that run KOI script packages on Cortex-Agent endpoints on a schedule, driven by two Jobs and configured through a JSON List, with per-endpoint scan-coverage tracking (section 10).
Alert triage, investigation & response playbooks	Seven playbooks that triage each KOI alert, run a full item and device investigation, compute a verdict, and drive analyst-gated response (section 11).

1.3 Pack capabilities at a glance

Capability	What you get
Telemetry collection	KOI Alerts and Audit logs stream into the koi_koi_raw dataset, normalized by the parsing rule and mapped to the Cortex Data Model by the modeling rule — queryable in XQL and visualized by the bundled dashboard.
Asset & posture visibility	Query the software your workforce actually runs — browser extensions, IDE plugins, npm/PyPI packages, desktop applications, MCP servers and other agentic-AI assets — per item or per device, with KOI's risk scoring.
Threat intelligence	The Koidex catalog returns an independent risk score, findings, compliance data and an AI-generated risk summary for any item, whether or not it is installed in your organization.
Automated alert triage	Every KOI alert is parsed, investigated and classified Benign / Suspicious / Malicious from KOI's own alert type and risk level plus the independent catalog risk. Benign alerts auto-close; the rest escalate for review.
Deep investigation	Per item: catalog risk, AI summary, organization exposure (installs, publisher, signing, endpoint count), the endpoints and users affected, governance state,

Capability	What you get
	and remediation/approval history. Per device: everything installed on the affected host and which of it is risky.
Analyst-gated response	A Malicious verdict opens a block approval showing the full investigation; the organization-wide blocklist write happens only after a human approves. Items already on the blocklist short-circuit.
Proactive hunting	A scheduled MCP-server audit flags risky agentic-AI assets across the estate without waiting for an alert.
Fleet script execution	Run KOI script packages (for example the deployment script) on Cortex-Agent endpoints on a schedule, configured entirely through a JSON List, with per-endpoint scan-coverage tracking so even endpoint groups larger than 100 are fully covered over successive runs.

1.4 What's new

- **Scan-coverage tracker (v1.4.0)** — the Script Runner is now tracker-driven: the new KoiScanTracker automation and a Refresh Job keep a per-endpoint ledger of when each endpoint was last scanned, so every endpoint is scanned once then re-scanned only after a configurable interval, and endpoint groups larger than 100 are fully covered over successive runs (section 10).
- **Alert triage, investigation and response playbooks** — automated triage with a full item and device investigation, a data-driven verdict, and an analyst-gated block (section 11).
- **13 read-only commands** covering devices, device inventory, findings, groups, users, remediations, approval requests, runtime (hardening) policies, Koidex catalog search and risk reports, and fetch-state diagnostics.
- **First Fetch Time Range** parameter to control the initial event-collection lookback window.
- **Tenant Name** parameter and automatic tenant identification: every collected event is stamped with koi_tenant_name and koi_customer_id, so events from multiple KOI tenants stay distinguishable.
- **Parsing Rule, Modeling Rule, and the Koi Alerts Dashboard** (new XSIAM content).

2. Prerequisites

- An active KOI tenant, and a KOI account with the xt-Administrator role (required to create API keys).
- A Cortex XSIAM tenant — event collection, the parsing and modeling rules, the dashboard and the command surface all run there.
- Outbound HTTPS access from your Cortex tenant/engine to `https://api.prod.koi.security/`.
- **An egress IP that KOI accepts.** KOI applies IP-based restrictions at its edge to the sensitive API endpoints (users, inventory, Koidex, governance). Requests leaving from shared datacenter or cloud egress ranges can be rejected with HTTP 403 even when the API key is valid and the same key succeeds from a corporate network. Event collection is unaffected. If your Cortex tenant egresses from a cloud range, run the KOI integration on a Cortex engine whose egress IP KOI accepts — see section 15.

3. Create Your KOI API Key

1. **Ensure the appropriate role.** To create an API key you must hold the xt-Administrator role in KOI.
2. **Open Settings.** Access the Settings page from the top navigation bar of the KOI console.
3. **Open the API Access tab.** In Settings, select the API Access tab.
4. **Create the key.** Click Create new API key.
5. **Copy the key.** Within a few seconds the new key appears in the table. Click Copy next to it and store it securely — you will paste it into the integration instance.

Treat the API key like a password: store it in a secret manager and rotate it according to your organization's policy.

4. Install the Pack

This pack is delivered to you directly — as pack source or a pack zip — and is not distributed through the Cortex Marketplace. Install only from the artifact you were sent.

Do not install the KOI pack published in the Cortex Marketplace. That is a different build produced by someone else; its contents and version do not match this pack.

1. Take the pack artifact you were sent (pack source, or a pack zip).
2. Upload it to the tenant with demisto-sdk — the full procedure is in section 12.
3. Confirm the integration, parsing and modeling rules, dashboard and playbooks all appear on the tenant.

If you only need the scheduled Script Runner Jobs and nothing from the KOI API, you can skip the pack install entirely and import six items by hand — see section 13.2.

5. Configure the Integration Instance

In Cortex XSIAM, go to Settings → Configurations → Automation & Feed Integrations. Search for KOI and click Add instance.

5.1 Connection parameters

Parameter	Description	Default
Server URL	The KOI API server URL.	https://api.prod.koi.security/
API Key	The key created in section 3.	—
Tenant Name	Optional label stamped on every collected event as koi_tenant_name (e.g., "Production"). The tenant UUID (koi_customer_id) is captured automatically from the API key.	—
Trust any certificate	Skip TLS verification (not recommended).	Off
Use system proxy settings	Route via the system proxy.	Off

5.2 Event-collection parameters (Cortex XSIAM only)

Parameter	Description	Default
Fetch events	Enables the event collector.	Off
Fetch event types	Which KOI event streams to collect: Alerts, Audit.	Alerts, Audit
Audit log type filter	Optionally restrict audit collection to selected types (approval_requests, devices, endpoints, extensions, firewall, guardrails, notifications, policies, remediation, requests, settings, vetting).	All types
First Fetch Time Range	Lookback window for the first fetch only (e.g., "3 days", "1 hour"). Afterwards the collector always resumes from the last-fetched event timestamp.	3 days
Maximum number of events per fetch	Cap per event type per fetch cycle.	5000
Events Fetch Interval	How often the collector runs.	1 minute

Multi-tenant tip: to collect from several KOI tenants, create one instance per tenant, each with its own API key and a distinct Tenant Name label — the koi_tenant_name field keeps the datasets cleanly separated.

6. Verify the Setup

1. Click Test on the instance. A successful test confirms URL and API key.
2. From the CLI / War Room, run a read-only command:

```
!koi-devices-list limit=5
```

3. Confirm a device table is returned. If you enabled event collection, allow one fetch interval and check the dataset (section 8).

7. Using the Commands

Commands are grouped below by what they operate on. Commands marked with * modify tenant state; all others are read-only. Most list commands support page/page_size for a specific page, or limit to auto-paginate.

7.1 Devices & organization

Command	Description
`koi-devices-list`	List devices registered with KOI (filter by status, last-seen, and more).
`koi-device-inventory-get`	List the items installed on a single device.
`koi-groups-list`	List device groups.
`koi-users-list`	List KOI users.

7.2 Inventory

Command	Description
`koi-inventory-list`	Paginated list of items installed across your endpoints, with rich filters (marketplace, platform, risk level, publisher, category).
`koi-inventory-item-get`	Full details for one item by ID + marketplace (+ optional version).
`koi-inventory-search`	Advanced query-builder search via filter JSON.
`koi-inventory-item-endpoints-list`	Endpoints that have a specific item installed.

7.3 Governance & policies

Command	Description
`koi-policy-list`	List all policies.
`koi-policy-status-update *`	Enable/disable a policy by ID.
`koi-runtime-policies-list`	List agent runtime (hardening) enforcement policies.
`koi-runtime-policy-get`	One runtime policy by ID, including its full rule tree.
`koi-findings-list`	List finding (detection) definitions — resolve finding IDs returned elsewhere.
`koi-approval-requests-list`	List approval requests.
`koi-remediations-list`	List remediation suggestions.

7.4 Allowlist & blocklist

Command	Description
`koi-allowlist-get / koi-blocklist-get`	Retrieve the global allowlist / blocklist.
`koi-allowlist-items-add * / koi-blocklist-items-add *`	Add one item (item_id + marketplace) or bulk-add from a JSON file entry.
`koi-allowlist-items-remove * / koi-blocklist-items-remove *`	Remove one item or bulk-remove from a JSON file entry.

7.5 Threat intelligence — Koidex catalog

Command	Description
`koi-koidex-search`	Search KOI's catalog database by marketplace + search term.
`koi-koidex-risk-report`	Risk + compliance report for a single catalog item.

7.6 Collector & diagnostics

Command	Description
`koi-get-events`	Manually pull events. Development/debugging only — keep should_push_events=false to avoid duplicates.
`koi-fetch-context-get`	Read-only diagnostic: prints the collector's fetch state (last run / high-water mark).
`koi-fetch-context-set *`	Maintenance: adjust the fetch state to recover a stalled collector.

7.7 Worked examples

List active devices:

```
!koi-devices-list limit=50 status=active
```

Everything installed on one device:

```
!koi-device-inventory-get device_id=DEVICE-123 limit=50
```

Find high-risk items across the fleet:

```
!koi-inventory-search filter_json="{\"field\":\"risk_level\",\"operator\":
\"eq\",\"value\":\"high\"}" limit=50
```

Vet a package before approving it:

```
!koi-koidex-search marketplace=npm search_term=axios
!koi-koidex-risk-report item_id=axios marketplace=npm
```

Block a malicious extension organization-wide:

```
!koi-blocklist-items-add item_id=malicious-ext marketplace=chrome_web_store
notes="Blocked due to security risk"
```

8. Event Collection into Cortex XSIAM

With Fetch events enabled, the integration collects two streams from KOI on every fetch interval:

- **Alerts** — KOI security alerts (OCSF-shaped Application Security Posture Findings).
- **Audit** — administrative and item activity (installs, updates, policy changes, remediations…).

Collection is incremental: the collector tracks the newest event timestamp per stream (a high-water mark) and resumes from it on every run, so events are neither missed nor duplicated. Events land in the koi_koi_raw dataset, where the pack's parsing rule normalizes them and the modeling rule maps them to XDM.

8.1 Useful XQL starting points

Recent KOI alerts:

```
dataset = koi_koi_raw | filter source_log_type = "Alerts" | sort desc _time |
limit 100
```

Audit activity by type:

```
dataset = koi_koi_raw | filter source_log_type = "Audit" | comp count() as
events by type | sort desc events
```

Volume per tenant and stream (multi-tenant deployments):

```
dataset = koi_koi_raw | comp count() as events by koi_tenant_name,
source_log_type
```

9. KOI Alerts Dashboard

The pack ships the "Koi Alerts Dashboard", which visualizes KOI alert activity from the koi_koi_raw dataset. After installing the pack in XSIAM, open Dashboards & Reports, locate Koi Alerts Dashboard, and add it to your dashboards. Data appears once event collection has run (section 8).

10. KOI Script Runner Playbooks

This set runs KOI script packages (for example, the KOI deployment script) on Cortex-Agent endpoints on a recurring schedule, configured entirely through a JSON List — retargeting or adding scripts never requires editing a playbook. Coverage is tracker-driven: a per-scope CSV List records when each endpoint was last scanned, so every endpoint is scanned once and then re-scanned only after a configurable interval, and an endpoint group larger than the 100-endpoint platform query cap is still covered fully across successive runs.

The system runs as two time-triggered Jobs sharing one configuration List. A Refresh Job (infrequent — e.g. hourly) keeps each scope's tracker List populated with its group's current membership; a Scan Job (frequent — e.g. every 10 minutes) dispatches the script to the endpoints that are due and connected, then marks them scanned. The KoiScanTracker automation owns the tracker List and all the due / connected / OS-safe / mark logic, so the playbooks stay thin.

10.1 Architecture

Playbook / Automation	Role
KOI Script Runner - Scan Job	Scan Job entry point. Loads the "Koi Script Runner" List and loops over its entries — one iteration per entry, each in an isolated context. Closes its own investigation when done.
KOI Script Runner - Process Config Entry	Per-entry logic: honors the disabled flag, validates the entry (invalid entries are reported in the war room, never silently dropped), invokes the executor, routes the outcome, sends the optional email notification, and cleans up.
KOI Script Runner - Execute Endpoint Script	Asks KoiScanTracker for up to max endpoints that are due, currently connected, and of the entry's OS; runs the script on them with polling; marks them scanned (mark-on-dispatch); returns a structured ScriptResult.
KOI Script Runner - Refresh Job	Refresh Job entry point. Loads the same "Koi Script Runner" List and loops over its entries to refresh each scope's tracker.
KOI Script Runner - Refresh Entry	Per entry: walks the endpoint group (paged past 100 via the Core REST API) and append-only upserts membership into the scope's tracker List — never touching an existing last_scan.
KoiScanTracker (automation)	Owns the tracker List I/O and the date/coverage logic. Action-dispatched: refresh (enumerate + upsert), select (return due,

Playbook / Automation	Role
	connected, right-OS ids), mark (stamp last_scan=now).

10.2 What must already exist on the tenant (not shipped in the pack)

The playbooks reference several objects strictly by name at runtime. None of these are delivered by the pack — they must be created on the tenant before the first run, with names that match the configuration exactly (case and spacing included):

Dependency	Where it lives	Requirement
Core REST API integration	Automation & Feed Integrations	Enabled. KoiScanTracker calls core-api-post through it to page an endpoint group past the 100-endpoint query cap during Refresh. Without it, Refresh cannot build the tracker and Scan has nothing to run.
KOI script package(s)	Action Center → Scripts Library	Upload the KOI deployment script(s) yourself — they are NOT part of this pack. The Library name must match the List's script.name exactly (e.g., "KOI Deployment Script - Windows"). The script must be parameterless — the run task passes no parameters_values. To make the reference rename-proof, pin script.uuid instead.
Endpoint group(s)	Endpoints → Endpoint Groups	Every group named in target.endpoint_groups must exist and contain the intended agents. The simplest way to build one is to tag the agents and create a dynamic group on that tag, so membership maintains itself as endpoints join and leave. Alternatively, scope by target.endpoint_hostnames.
Target agents	Endpoints	Cortex agents installed on the targets, CONNECTED and not isolated, with an OS matching the entry's endpoint_os — otherwise they are simply not selected this run and stay due.
Configuration List	Settings → Object Setup → Lists	A JSON List named exactly "Koi Script Runner" (section 10.3).
Tracker List(s)	Settings → Object Setup → Lists	One CSV List per scope, named by each entry's target.tracker_list. Created automatically by the first Refresh run — no manual step.
Mail-sender instance (optional)	Automation & Feed Integrations	Required only when notifications are configured: an enabled instance that supports send-mail. If notification.sendmail_instance.name is set, an instance with that exact name must exist.

Everything binds by name at run time: renaming a script in Action Center, an endpoint group, the List, or a mail instance — without updating the List configuration — causes the next run to fail or skip (the war-room reason states which reference broke).

10.3 Configuration — the "Koi Script Runner" List

Create a JSON List named exactly "Koi Script Runner" (Settings → Configurations → Object Setup → Lists) containing an array of entries:

```
[
  {
    "disabled": false,
    "script": { "name": "KOI Deployment Script - Windows" },
    "target": {
      "endpoint_groups": ["KOI Endpoints"],
      "endpoint_hostnames": [],
      "endpoint_os": "Windows",
      "tracker_list": "Koi Scan Tracker - Windows",
      "rescan_interval_hours": 720
    },
    "notification": {
      "sendmail_instance": { "name": "internal-smtp" },
      "recipients": ["ops@example.com"]
    }
  }
]
```

Field	Required	Description
disabled	No	Set true to skip the entry entirely.
script.name	Yes*	Exact script name in the Action Center Script Library. The script must be parameterless. (*Either name or uuid.)
script.uuid	No	Script UUID — takes precedence over name and disambiguates duplicate names.
script.polling_interval_in_seconds	No	Poll cadence while the script runs (default 60).
script.timeout_in_seconds	No	Overall execution timeout (default 1800).
target.endpoint_os	Yes	Windows, macOS, or Linux (case-insensitive).
target.endpoint_groups	One of these	Endpoint group names to scope execution.
target.endpoint_hostnames	One of these	Specific hostnames to scope execution.
target.tracker_list	Yes	Name of the CSV List this scope's scan coverage is tracked in (e.g. "Koi Scan Tracker - Windows"). Refresh creates it if missing.
target.rescan_interval_hours	No	Hours before a scanned endpoint becomes due again (default 720 = 30 days).
target.max_endpoints	No	Endpoints dispatched to per Scan run (default 100, the

Field	Required	Description
		platform maximum for the connected-check).
notification.recipients	No	Email recipients for the per-entry outcome; omit for no email.
notification.sendmail_instance.name	No	Specific mail-sender instance; omit to use any enabled one.

10.4 How scripts are matched to endpoints

The pairing of script to operating system is declared per entry and enforced when endpoints are selected: KoiScanTracker returns only endpoints that are due (never scanned, or last scanned longer ago than rescan_interval_hours), currently connected and unisolated, AND of the entry's endpoint_os. A Windows entry therefore can never execute on a Mac even if a wrong-OS row somehow lands in its tracker — and one shared group can serve every OS, with each scope's tracker holding only its own members.

10.5 Job setup — two Jobs

Create two time-triggered Jobs (Investigation & Response → Automation → Jobs → New Job). Schedule them at non-overlapping times; Refresh is append-only and Scan only updates existing rows, so an occasional overlap self-heals on the next cycle.

1. **Refresh Job.** Time-triggered, infrequent (e.g., hourly), playbook **KOI Script Runner - Refresh Job**. Run this at least once before the first Scan so the tracker is populated.
2. **Scan Job.** Time-triggered, frequent (e.g., every 10 minutes), playbook **KOI Script Runner - Scan Job**. Save, then use Run now for an immediate first run.
3. **Enable both Jobs.** A newly created Job can be disabled by default. Confirm the scheduling toggle is on and a next-run time is shown for each — otherwise the Job never fires, the tracker stays empty, and Scan has nothing to do.

10.6 Run outcomes

Outcome	Meaning	Notification
ok: true (+ action_id)	Script dispatched to the due, connected endpoints; action_id links to Action Center. Marked scanned on dispatch — the run does not wait for Action Center to finish.	Success email (when recipients configured).
skipped: true	Entry valid, but no due, connected endpoint currently matches the scope — nothing to do this run. Logged as an info entry.	None — keeps recurring jobs quiet for scopes with nothing due.
timed_out: true	The action was dispatched, but the playbook's poll expired before it finished. Not a failure — the endpoint is already marked scanned; a STILL RUNNING entry is logged.	None.
ok: false + reason	Real failure: script not found, ambiguous	Failure email (when recipients

Outcome	Meaning	Notification
	name, tracker/endpoint query error, or a dispatch failure. The reason states which.	configured).

11. Alert Triage, Investigation & Response Playbooks

A second, independent playbook set turns each KOI alert into an investigated, classified and — when you approve it — actioned case. Attach KOI IR - Alert Triage to your KOI alerts and the rest runs automatically.

11.1 The playbooks

Playbook	Role
KOI IR - Alert Triage	Main. Extracts the alert context, runs the item and device investigations, computes a verdict, posts a summary, and routes: auto-close benign, escalate the rest, and open an analyst-gated block for Malicious.
KOI IR - Extract Alert Context	Sub. Parses the koi_context JSON embedded in the alert description into the KoiContext object.
KOI IR - Investigate Item	Sub. Full investigation of the flagged item (11.3).
KOI IR - Investigate Device	Sub. Full investigation of the affected device (11.4).
KOI IR - Enrich Item	Sub. Lightweight enrichment (catalog risk plus endpoint exposure) for callers that do not need the full investigation.
KOI IR - Block and Remediate	Investigates the item, skips it if already blocked, then adds it to the organization blocklist only after analyst approval.
KOI Hunting - MCP Server Audit	Scheduled hunt. Flags MCP servers at or above a risk threshold; attach to a time-triggered Job.

11.2 Triage flow and verdicts

The verdict is keyed on KOI's own alert type and risk level, with the independent Koidex catalog risk as a corroborating signal:

Verdict	Condition	Action
Malicious	Alert type is Removed from Marketplace, Publisher Compromised or Unvetted MCP Server; or KOI risk level is High or Critical; or the catalog risk is high/critical.	Raise severity, then open an analyst-gated block for the item.
Benign	Alert type is New Item and KOI risk level is Low.	Auto-close the alert as Resolved.
Suspicious	Anything else (safe default).	Leave open for analyst review.

11.3 What the item investigation gathers

- Catalog risk score and level, plus KOI's AI-generated risk summary.

- Your organization's own inventory record for the item: installs count, endpoint count, publisher, signing status and organization risk level.
- The endpoints the item is installed on and the users affected (resolved when the installed version is known).
- Governance state — whether the item already appears on the organization blocklist.
- Remediation and approval history filtered to that item.

11.4 What the device investigation gathers

- Every item installed on the affected device, with version, marketplace, publisher and install path.
- Which of those items are at or above the configured risk level (default: high, critical), posted as a risky-items table.
- Remediations recorded against that host.

11.5 Response safety

Adding an item to the blocklist is the only state-changing action in this set, and it is gated. KOI IR - Block and Remediate presents the full investigation to an analyst and writes only on approval; the triage always calls it with auto_block set to false, so a Malicious verdict can never block software without a human decision. Setting auto_block to true (only meaningful when you invoke the playbook directly) skips the gate — use with care.

11.6 Attaching the triage

1. Ingest KOI alerts into Cortex (section 8) so each alert carries its koi_context payload.
2. Attach **KOI IR - Alert Triage** as the playbook for KOI alerts, or run it on an existing alert from the CLI:

```
!setPlaybook playbookId="KOI IR - Alert Triage"
```

3. Confirm in the war room: an item investigation summary, a device investigation summary, and a triage summary carrying the verdict.

KOI IR - Investigate Item, KOI IR - Investigate Device and KOI IR - Block and Remediate declare required inputs, so they are designed to be called as sub-playbooks (or invoked by an analyst supplying those inputs) rather than attached directly to an alert.

12. Deploying the Full Pack as a Single Object (demisto-sdk)

The pack can be delivered to a tenant as one artifact — a pack zip — using Palo Alto Networks' demisto-sdk. This is the path to use for every tenant, since the pack is delivered directly rather than through the Cortex Marketplace.

12.1 Choosing an install path

The path you choose determines whether event collection works at all. Commands and playbooks behave the same on every path; the collector and the rules do not.

Path	What lands on the tenant	Events flow?	Use when
demisto-sdk --xsiam	Everything, at the version you were sent	Yes	The normal path
Pre-built dist/Koi.zip	Integration plus an old 3-playbook Script Runner only — no tracker, no rules, no dashboard	No	Not for XSIAM
Manual per-item	Only the items you import yourself	Only if you add the rules	Partial adoption

The pre-built dist/Koi.zip was built for a different marketplace target. Inside it the integration carries isfetchevents: false, and the artifact contains no parsing rules, no modeling rules and no dashboard. Uploading it to a Cortex XSIAM tenant produces an instance whose commands all work while the koi_koi_raw dataset stays permanently empty — which presents like a collector fault but is simply the wrong artifact. For working event collection, upload with demisto-sdk --xsiam.

12.2 One-time setup

1. **Install the SDK:** `pip3 install demisto-sdk` (1.38+ required — older versions reject packs that declare the platform marketplace).
2. **Place the pack in a content-repo structure.** demisto-sdk only runs inside a repo shaped like demisto/content: a git repository containing Packs/Koi/<pack contents>, an empty .private-repo-settings file at the root, and Tests/secrets_white_list.json containing {"iocs":[],"files":[],"generic_strings":[]}.
3. **Set the connection environment variables:** `DEMISTO_BASE_URL` (the api- URL of the tenant), `DEMISTO_API_KEY`, and `XSIAM_AUTH_ID` (the key's ID).

12.3 Build and push

Validate, build the single pack artifact, and upload:

```
demisto-sdk validate -i Packs/Koi/Playbooks
demisto-sdk zip-packs -i Packs/Koi -o zipped
demisto-sdk upload -i Packs/Koi -z --xsiam
```

12.4 Notes verified in practice

- **API key type matters.** demisto-sdk sends the API key as-is, which works only with a Standard XSIAM API key. With an Advanced key the SDK cannot connect; in that case build the artifact with zip-packs and POST zipped/uploadable_packs/Koi.zip (multipart, field name "file") to https://api-<tenant>/xsoar/contentpacks/installed/upload?skipVerify=true&skipValidation=true using the advanced-key signed headers. Note the endpoint is under /xsoar/, not /xsoar/public/v1/.
- **API modules.** The Koi integration imports ContentClientApiModule; the build inlines it from Packs/ApiModules/Scripts/ContentClientApiModule/ — copy that folder from the demisto/content repository into your content repo before running zip-packs.
- **Pack items become system-owned.** After a pack install, its items (playbooks, integration) are marked system and reject individual item uploads. Ship subsequent changes as a new pack version (bump currentVersion in pack_metadata.json, add a release note, re-upload the zip).

After upload, the pack appears in the tenant at the new version and every item (integration, playbooks, rules, dashboard) is installed together — see section 14 to validate.

13. Onboarding Individual Items Manually

If you prefer to adopt only part of the pack (or cannot use the SDK), each item can be onboarded by hand in the tenant UI. Import order matters where items reference each other by name.

13.1 Item-by-item reference

Item	How to onboard manually
Integration (Koi.yml)	Settings → Configurations → Automation & Feed Integrations → Upload Integration, and select the YAML. Note this installs the integration only — the parsing and modeling rules and the dashboard are separate items below.
Automation — KoiScanTracker	Investigation & Response → Automation → Scripts → Import. Import this before the Script Runner playbooks — they all call it by name.
Playbooks — Script Runner (5 files)	Investigation & Response → Automation → Playbooks → Import. Import in dependency order (child before parent): 1) KOI Script Runner - Execute Endpoint Script, 2) KOI Script Runner - Process Config Entry, 3) KOI Script Runner - Refresh Entry, 4) KOI Script Runner - Refresh Job, 5) KOI Script Runner - Scan Job — references bind by name.
Playbooks — Triage & response (7 files)	Same import screen. Import the sub-playbooks first (KOI IR - Extract Alert Context, KOI IR - Investigate Item, KOI IR - Investigate Device, KOI IR - Enrich Item), then KOI IR - Block and Remediate and KOI Hunting - MCP Server Audit, and finally KOI IR - Alert Triage. References bind by name.
Configuration List	Settings → Configurations → Object Setup → Lists → New List. Name it exactly "Koi Script Runner", type JSON, and paste the configuration array (section 10.3).
Jobs (two)	Investigation & Response → Automation → Jobs → New Job, time-triggered: a Refresh Job on "KOI Script Runner - Refresh Job" (infrequent), and a Scan Job on "KOI Script Runner - Scan Job" (frequent). See section 10.5.
Parsing / Modeling Rules	XSIAM: Settings → Data Management → Parsing Rules (and Modeling Rules) → add the content of the pack's .xif files as user-defined rules targeting dataset koi_koi_raw.
Koi Alerts Dashboard	Dashboards & Reports → New Dashboard → Import, select Koi_Alerts_Dashboard.json.

Keep item names unchanged when importing manually — the Job references the main playbook by name, the main playbook references the List by name, and the sub-playbooks reference each other by name.

13.2 Running only the Script Runner Jobs

A common case is wanting nothing from KOI's API — only the ability to run KOI script packages on Cortex-Agent endpoints on a schedule. That needs a small subset of the pack.

The Script Runner playbooks invoke no koi-* command at all. They use only the Cortex-native core-get-scripts, core-get-endpoints, core-script-run and core-api-post, plus the built-in send-mail for

notifications. Consequently this deployment requires no KOI API key, no KOI integration instance, no parsing or modeling rules and no dashboard. The one integration it does need is the built-in Core REST API (for core-api-post, used by Refresh to page endpoint groups past 100).

Import exactly these six content items, in this order (import the automation first, then each playbook before the one that calls it — references bind by name):

#	Item	Where
1	KoiScanTracker (automation)	Investigation & Response → Automation → Scripts → Import
2	KOI Script Runner - Execute Endpoint Script	Investigation & Response → Automation → Playbooks → Import
3	KOI Script Runner - Process Config Entry	Investigation & Response → Automation → Playbooks → Import
4	KOI Script Runner - Refresh Entry	Investigation & Response → Automation → Playbooks → Import
5	KOI Script Runner - Refresh Job	Investigation & Response → Automation → Playbooks → Import
6	KOI Script Runner - Scan Job	Investigation & Response → Automation → Playbooks → Import

Then create the JSON List "Koi Script Runner" (Settings → Configurations → Object Setup → Lists → New List, type JSON) and two time-triggered Jobs: a Refresh Job on "KOI Script Runner - Refresh Job" (infrequent) and a Scan Job on "KOI Script Runner - Scan Job" (frequent). Enable both Jobs after creating them — confirm each shows a next-run time, or it will not fire. Run Refresh once before the first Scan; the per-scope tracker Lists are created automatically by that first Refresh run.

Enable the built-in Core REST API integration, and make sure these already exist on the tenant, referenced by name from the List: the parameterless KOI script package in Action Center → Scripts Library (its Library name must equal script.name exactly, or pin script.uuid instead), an endpoint group named in target.endpoint_groups containing connected and unisolated agents whose OS matches target.endpoint_os (tag the agents and use a dynamic group — it is the least maintenance), and — only if you configure notifications — an enabled mail-sender instance matching notification.sendmail_instance.name.

To validate this deployment on its own, run only test 9 of the test guide: Run now on the Refresh Job, then Run now on the Scan Job, and confirm the tracker List fills and ScriptResult ok:true with an action_id per scanned scope. Tests 1 to 8 all exercise the KOI API and do not apply.

14. Validating That Everything Works

Run these checks after any deployment, whether you uploaded the pack with the SDK or imported items by hand. Each row gives the action and the evidence that proves the component is healthy.

Component	How to validate	Expected evidence
API connectivity	Instance → Test button; then run !koi-devices-list limit=5 in the CLI.	Test returns success; a device table renders.
Event collector	Wait one fetch interval, then run the XQL	Recent rows with source_log_type

Component	How to validate	Expected evidence
	query below (a).	Alerts/Audit; count grows between runs.
Parsing & modeling rules	Inspect the XQL result fields.	Fields like source_log_type, alert_type, severity, mcp_* are populated (not only _raw_log).
Script Runner playbooks	Jobs → select the job → Run now; open the run.	Work Plan is all green; war room shows ScriptResult ok:true with an action_id per executed entry, and SKIPPED info entries for OS scopes with no endpoints.
Endpoint execution	Action Center → All Actions → find the action_id.	Status COMPLETED_SUCCESSFULLY on the expected endpoints.
Notifications	Configure a mail-sender instance; re-run the job.	Success/failure email arrives at the List's recipients; no send-mail errors in the war room.
Alert triage playbooks	Run KOI IR - Alert Triage on a KOI alert (attach it, or !setPlaybook).	War room shows an item investigation summary, a device investigation summary, and a triage summary carrying the verdict; benign alerts auto-close, others escalate.
Analyst-gated block	Let a Malicious verdict reach the block step.	The run parks at runStatus "waiting" on the approval task and koi-blocklist-items-add has NOT executed. It runs only after an analyst approves.
Dashboard	Open Koi Alerts Dashboard.	Widgets render data after the collector has ingested alerts.

(a) Collector validation query:

```
dataset = koi_koi_raw | sort desc _time | limit 10
dataset = koi_koi_raw | comp count() as events by source_log_type
```

Failure-path checks worth running once: temporarily set "disabled": true on a List entry (that entry must be skipped silently), and misspell a script name (the run must report a clear "not found" reason rather than hang).

15. Troubleshooting

Symptom	Likely cause	Resolution
"Authorization Error: Verify your API Key"	Key invalid, revoked, or pasted with whitespace	Re-create the key (Settings → API Access, xt-Administrator role) and update the instance.
Test succeeds but no events arrive	Fetch events disabled; wrong event types; or a stalled fetch state	Enable Fetch events and check Fetch event types. Run koi-fetch-context-get to inspect the high-water mark; if it is

Symptom	Likely cause	Resolution
		stuck (e.g., in the future), recover with koi-fetch-context-set.
koi-inventory-search returns a 400 error	filter_json is not a valid query-builder object	Provide filter JSON in the documented structure (field/operator/value rules); see the command reference for examples.
Koidex commands return a 400 error	Missing required arguments	koi-koidex-search requires both marketplace and search_term; koi-koidex-risk-report requires item_id and marketplace.
Duplicate events after manual pulls	koi-get-events was run with should_push_events=true	Use koi-get-events for debugging only, and keep should_push_events=false.
HTTP 429 / rate limiting	Fetch volume too aggressive	Lower Maximum number of events per fetch or increase the fetch interval.
HTTP 403 on inventory, users, Koidex or governance commands, while event collection keeps working	KOI's edge is rejecting the source IP — typically a shared datacenter or cloud egress range	Run the integration on a Cortex engine whose egress IP KOI accepts (for example a corporate/office network), or ask KOI to allowlist your egress. Confirm with !koi-users-list limit=1 — the same key succeeds from an accepted IP and returns 403 from a rejected one.

16. Support & References

- KOI product documentation: `https://docs.koi.ai`
- Pack release notes: see the ReleaseNotes folder inside the pack artifact you were sent.
- For issues with the integration content, contact your Palo Alto Networks / KOI support channel.